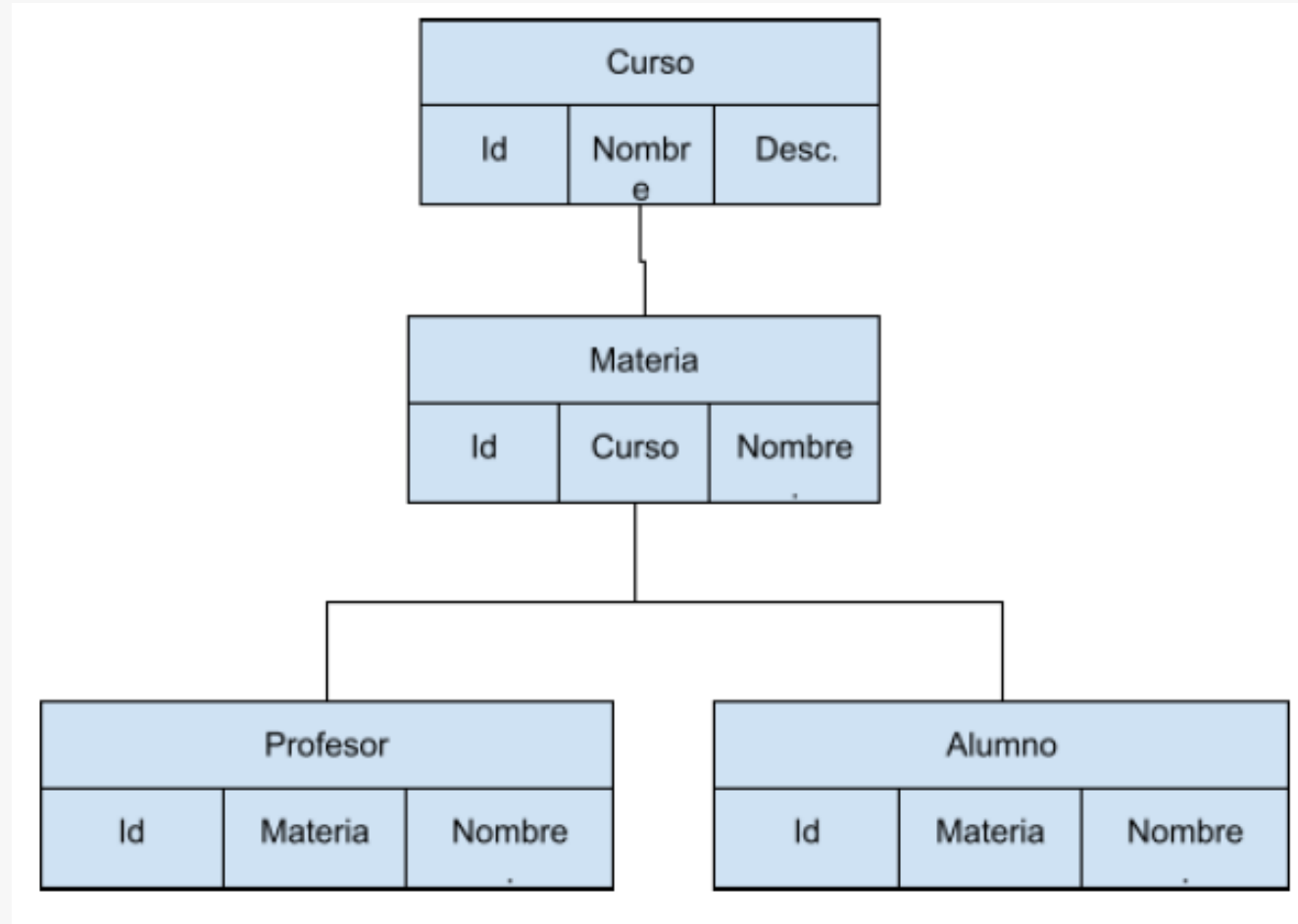


Base de Datos I

Modelo jerárquico y modelo de red. Consultas SQL avanzadas.

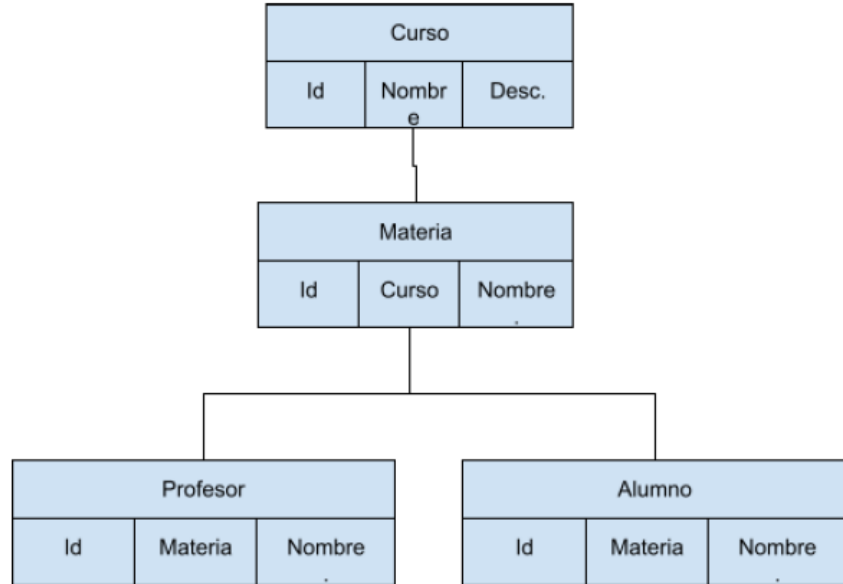
Modelo Jerárquico

Un sistema de base de datos jerárquico, es aquel que guarda la información en una estructura jerárquica



Características

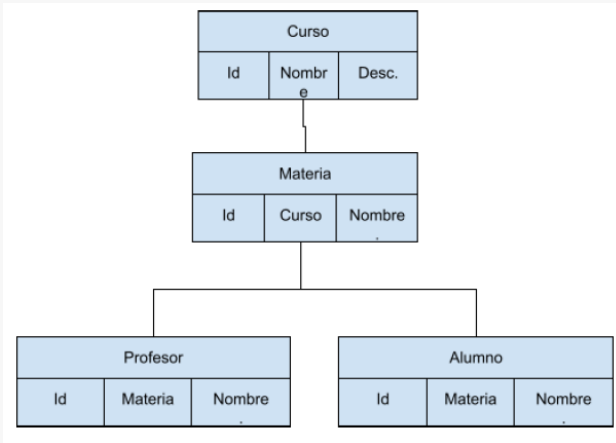
El modelo jerárquico permite implementar relaciones *padre-hijo*, que es una mejora presentada frente a los sistemas basados en archivos.



Los datos pueden recorrerse siguiendo las relaciones entre padres e hijos. La principal restricción es que cada nodo hijo solo puede pertenecer a un único nodo padre

Las relaciones entre los datos se implementan mediante mecanismos de navegación física, como punteros o referencias. Esto vincula la estructura lógica del árbol con la forma en que los registros se encuentran y se recorren físicamente.

Desventajas



Duplicidad de registros

- Esto en parte se debe a que este tipo de modelo no implementa el concepto de "clave primaria" ni tampoco el de "campos únicos".

Integridad referencial

- Es posible borrar un nodo *padre* sin eliminar antes los nodos *hijo*.

Desnormalización

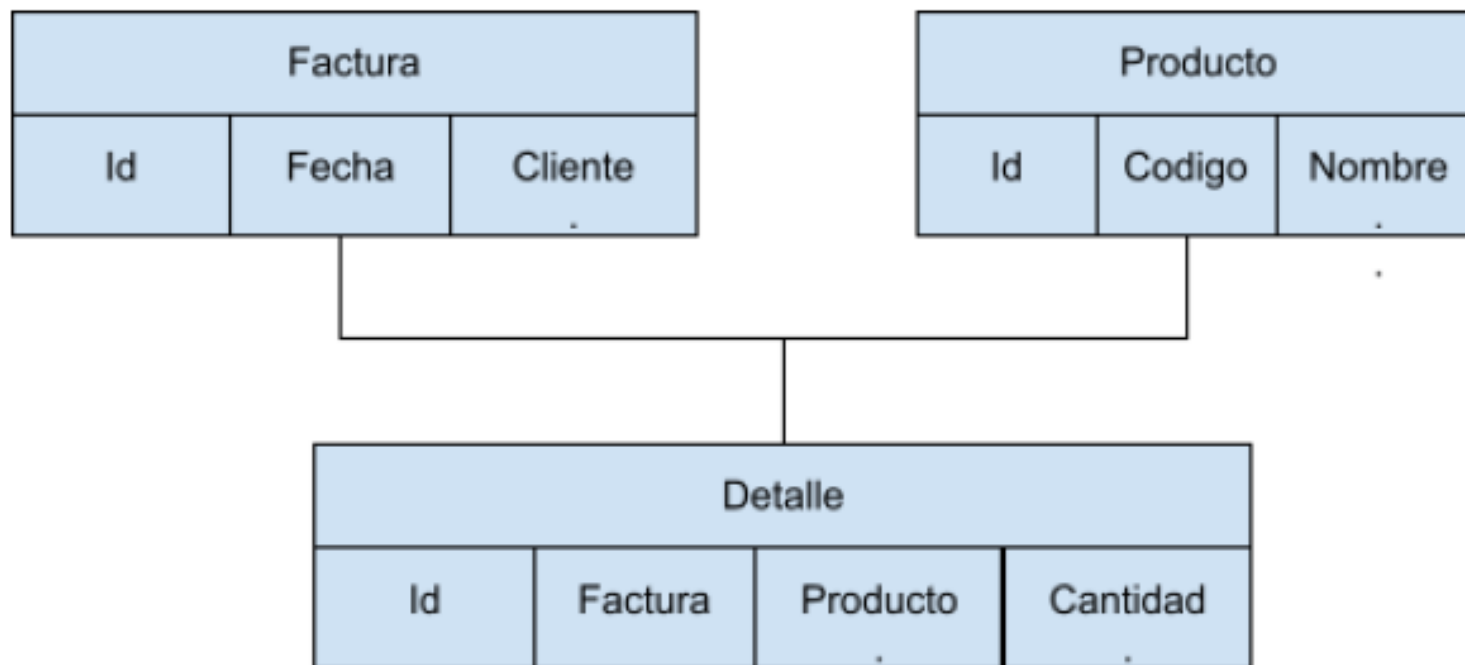
- No hay controles que impidan la desnormalización de una base de datos. La desnormalización de un modelo se debe a la redundancia de datos

Mantenibilidad

- Difícil de mantener ya que si quisiéramos implementar una nueva relación, probablemente todo el modelo se vea afectado

Modelo de Red

Un sistema de base de datos de red es aquel que guarda una colección o set de registros, conectados entre sí por medio de enlaces como una red.



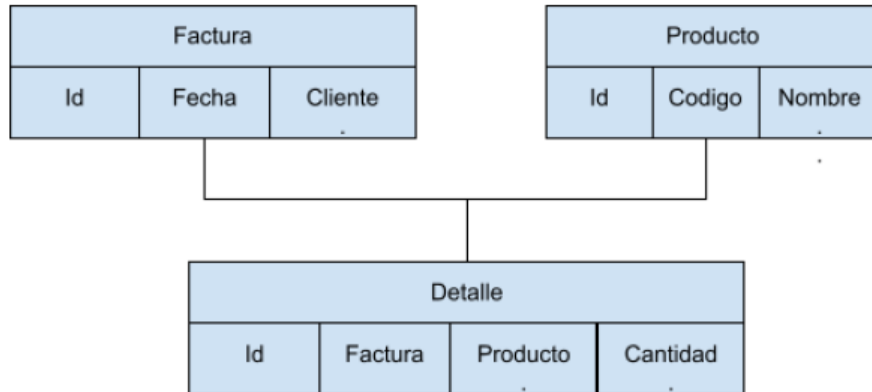
Características

Las relaciones o conjuntos de registros son denominadas *sets*. Los sets y los registros deben ser descritos en el esquema de la base de datos.

Un set incluye un único registro de un tipo. Este registro es denominado *owner* (dueño) de la relación

Un set incluye 0 (cero), 1 (uno) o más registros de un segundo tipo. Estos registros son denominados (members) miembros de la relación

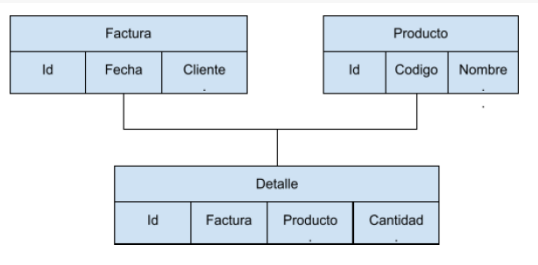
Los sets están ordenados.



Ventajas y desventajas del modelo de red

Ventajas

- Permite manejar más tipos de relaciones que el modelo jerárquico
- Es más flexible que el modelo jerárquico, aunque no todos los problemas se pueden resolver agregando "padres" a las relaciones.



Desventajas

- Para la navegación de los datos hay que conocer todo el modelo
- La curva de aprendizaje del lenguaje para consultar los datos es muy grande
- Los cambios estructurales son difíciles debido a que la implementación de las relaciones es física y no lógica.
- Un cambio en la estructura de las relaciones demandará un cambio en el código de la aplicación

DQL

Comando	Sintaxis	Resultado
GROUP BY	SELECT id_departamento, COUNT(*) AS cantidad FROM empleados GROUP BY id_departamento;	Agrupar registros para usar funciones agregadas
GROUP BY, HAVING	SELECT id_departamento, AVG(salario) AS promedio FROM empleados GROUP BY id_departamento HAVING AVG(salario) > 50000;	Solo muestra departamentos con promedio mayor a 50,000
JOIN	SELECT empleados.nombre, departamentos.nombre_departamento FROM empleados JOIN departamentos ON empleados.id_departamento = departamentos.id_departamento;	Combinar datos de varias tablas (SELECT, FROM, JOIN, ON)
JOIN (con alias)	SELECT e.nombre, d.nombre_departamento FROM empleados e JOIN departamentos d ON e.id_departamento = d.id_departamento;	Asigna temporalmente alias a empleados

DQL

Comando	Sintaxis	Resultado
INNER JOIN	SELECT e.nombre, d.nombre_departamento FROM empleados e INNER JOIN departamentos d ON e.id_departamento = d.id_departamento;	Devuelve empleados con su departamento, solo si existe en ambas tablas.
LEFT JOIN	SELECT e.nombre, d.nombre_departamento FROM empleados e LEFT JOIN departamentos d ON e.id_departamento = d.id_departamento;	Devuelve todas las filas de la primera tabla (izquierda) y, si hay coincidencia en la segunda, la muestra; si no, pone NULL.
RIGHT JOIN	SELECT e.nombre, d.nombre_departamento FROM empleados e RIGHT JOIN departamentos d ON e.id_departamento = d.id_departamento;	Devuelve todas las filas de la tabla derecha y, si hay coincidencia en la primera, la muestra; si no, pone NULL.
FULL OUTER JOIN	SELECT e.nombre, d.nombre_departamento FROM empleados e FULL OUTER JOIN departamentos d ON e.id_departamento = d.id_departamento;	Devuelve todas las filas de ambas tablas, poniendo NULL donde no hay coincidencia.

DQL

Comando	Sintaxis	Resultado
SELECT, DISTINCT	SELECT DISTINCT id_departamento, nombre FROM empleados;	Sirve para eliminar filas duplicadas en los resultados de una consulta. Devuelve pares únicos (id_departamento, nombre)
SELECT, DISTINCT	SELECT COUNT(DISTINCT id_departamento) AS total_departamentos FROM empleados;	Dice cuántos departamentos distintos hay en empleados

DISTINCT afecta a todas las columnas del SELECT.
No se puede usar con GROUP BY en las mismas columnas (pero se usan para objetivos similares).
Puede combinarse con ORDER BY y LIMIT.

iGracias!

Centro de
e-Learning



UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES